Lecture 1: Kronecker Products, Sum Factorization, and Tensor Contractions

1. Definition of the Kronecker Product

Definition: Let $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{p\times q}$. The Kronecker product $A \otimes B \in \mathbb{R}^{mp\times nq}$ is defined as the block matrix:

$$A \otimes B = \begin{bmatrix} a_{11}B & a_{12}B & \dots & a_{1n}B \\ a_{21}B & a_{22}B & \dots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \dots & a_{mn}B \end{bmatrix}$$

Key Properties (Mixed-Product Property): For matrices of compatible dimensions,

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$$

$$(A \otimes B)^T = A^T \otimes B^T$$

$$(A \otimes B)^{-1} = A^{-1} \otimes B^{-1} \quad \text{(if } A, B \text{ are invertible)}$$

2. Dimensional Splitting

In FEM on tensor-product elements (e.g., hexahedra), operations like the mass matrix or evaluation of basis functions decompose along spatial dimensions.

Let an operator on a 3D Cartesian cell be defined as $M = A \otimes B \otimes C$, where $A, B, C \in \mathbb{R}^{N\times N}$ are 1D operators (e.g., 1D mass matrices). N is the number of 1D Degrees of Freedom ($N = p + 1$, where p is the polynomial degree).

The full operator $M \in \mathbb{R}^{N^3\times N^3}$. We can split the application of M using the mixed-product property.

Proof of the split: Note that the identity matrix of size N is I. We can rewrite A, B, C as:

$$A = A \cdot I \cdot I$$

$$B = I \cdot B \cdot I$$

$$C = I \cdot I \cdot C$$

Using the mixed-product property:

$$A \otimes B \otimes C = (AII) \otimes (IBI) \otimes (IIC)$$

$$A \otimes B \otimes C = (A \otimes I \otimes I)(I \otimes B \otimes I)(I \otimes I \otimes C)$$

This shows that applying the 3D operator M to a vector $u \in \mathbb{R}^{N^3}$ can be done sequentially: 1. $v^{(1)} = (I \otimes I \otimes C)u$ *(operate on the x-direction)* 2. $v^{(2)} = (I \otimes B \otimes I)v^{(1)}$ *(operate on the y-direction)* 3. $v^{(3)} = (A \otimes I \otimes I)v^{(2)}$ *(operate on the z-direction)*

3. Computational Complexity Analysis

Let's evaluate $v = Mu = (A \otimes B \otimes C)u$.

Approach 1: Naive (Standard Matrix-Vector Multiply)

Assemble $M = A \otimes B \otimes C$ explicitly. - M is size $N^3 \times N^3$. - Dense matrix-vector multiplication requires $\mathcal{O}((N^3)^2) = \mathcal{O}(N^6)$ operations. - **Exact FLOP count:** $2N^6$ (1 multiply + 1 add per element in the dot product).

Approach 2: Sum Factorization (Dimensional Split)

Evaluate v sequentially via $v^{(1)}, v^{(2)}, v^{(3)}$. - Evaluating $v^{(1)} = (I \otimes I \otimes C)u$ means applying the $N \times N$ matrix C to N^2 independent 1D vectors of length N. - Operations for step 1: $N^2 \times (2N^2) = 2N^4$ FLOPs. - Since steps 2 and 3 do the exact same amount of work along different dimensions, the total operations are $3 \times 2N^4 = 6N^4$. - Complexity: $\mathcal{O}(N^4)$.

Comparison and Practical Limitations

Let's compare exact FLOP counts for different polynomial degrees p (where $N = p + 1$):

Degree p	1D Size N	Naive ($2N^6$)	Sum Fact. ($6N^4$)	Ratio (Naive / SF)
$p = 1$	2	128	96	1.33
$p = 2$	3	1,458	486	3.00
$p = 3$	4	8,192	1,536	5.33
$p = 4$	5	31,250	3,750	8.33

Important Note for Implementation: For $p = 1$ ($N = 2$), the theoretical FLOP advantage is marginal (96 vs 128). In practice, sum factorization requires complex loop structures (strided memory access), whereas the naive approach is a single contiguous dense matrix-vector multiply (BLAS Level 2). Due to instruction latency, loop overhead, and cache prefetching, the **naive assembled method is often faster for** $p = 1$. Sum factorization becomes strictly necessary and dominant for $p \geq 2$.

4. Einstein Summation and Tensor Representation

We can map the 1D flat vector $u \in \mathbb{R}^{N^3}$ to a 3D tensor $U \in \mathbb{R}^{N \times N \times N}$. Let U_{ijk} represent the degree of freedom at node (i, j, k).

Einstein Summation Convention: Repeated indices in a single term imply summation over that index.

Let's rewrite the sequential sum factorization using tensor index notation. 1. Apply C to the fastest changing index k (x-direction):

$$V^{(1)}_{ijm} = \sum_{k=1}^{N} C_{mk} U_{ijk} \quad \xrightarrow{\text{Einstein}} \quad C_{mk} U_{ijk}$$

2. Apply B to index j (y-direction):

$$V^{(2)}_{inm} = B_{nj} V^{(1)}_{ijm} = B_{nj} C_{mk} U_{ijk}$$

3. Apply A to index i (z-direction):

$$V^{(3)}_{lnm} = A_{li} V^{(2)}_{inm} = A_{li} B_{nj} C_{mk} U_{ijk}$$

The full evaluation of the Laplace operator (or mass matrix) on a single Cartesian cell is beautifully compressed to:

$$V_{lnm} = A_{li} B_{nj} C_{mk} U_{ijk}$$

5. Implementation Overview with JAX / NumPy

Because the operation is purely multidimensional tensor contraction, it maps perfectly to `einsum`.

See the runnable example in examples/A01_einsum.py.

6. Exercise: 2D Kronecker Product and the Transpose Trick

Problem: Let $u \in \mathbb{R}^{N^2}$ be a vector, and let $A, B \in \mathbb{R}^{N \times N}$. Show that computing $v = (A \otimes B)u$ is mathematically equivalent to computing:

$$V_{out} = BUA^T$$

where $U \in \mathbb{R}^{N \times N}$ is the tensor (matrix) representation of u, and V_{out} is the tensor representation of v.

Proof Outline:

1. **Element-wise definition:** Let's write out the matrix multiplication BUA^T. The (i, j)-th element of V_{out} is given by:

$$(V_{out})_{ij} = \sum_{k=1}^{N} \sum_{l=1}^{N} B_{ik} U_{kl} (A^T)_{lj}$$

   Since $(A^T)_{lj} = A_{jl}$, we have:

$$(V_{out})_{ij} = \sum_{k=1}^{N} \sum_{l=1}^{N} B_{ik} A_{jl} U_{kl}$$

2. **Index Mapping:** Assume column-major ordering (vectorization by stacking columns). The 2D indices (k, l) of U map to the 1D index of u via: $I = k + (l - 1)N$. The 2D indices (i, j) of V_{out} map to the 1D index of v via: $J = i + (j - 1)N$.

3. **Connecting to Kronecker:** By definition of the Kronecker product $A \otimes B$, the block at row block j, column block l is $A_{jl}B$. The specific element in that block at local row i and local column k is exactly $A_{jl}B_{ik}$.

   Therefore, the matrix-vector product $v = (A \otimes B)u$ at flat index J is:

$$v_J = \sum_{I=1}^{N^2} (A \otimes B)_{JI} u_I$$

   Substituting the 2D indices:

$$v_{i+(j-1)N} = \sum_{l=1}^{N} \sum_{k=1}^{N} (A_{jl}B_{ik}) U_{kl}$$

   This perfectly matches the element-wise expansion of BUA^T.

Takeaway: Matrix-matrix multiplication is implicitly doing sum-factorization in 2D! * UA^T applies A along the x-direction (rows of A hit columns of U). * $B(UA^T)$ applies B along the y-direction (rows of B hit columns of the intermediate matrix).